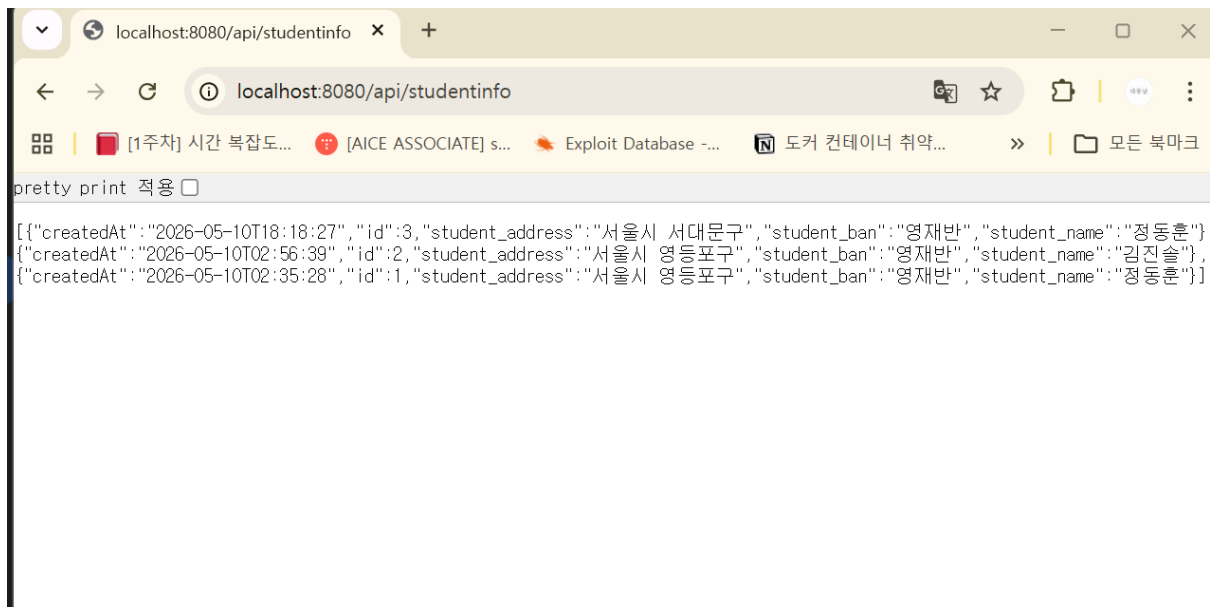


# 자습 과제

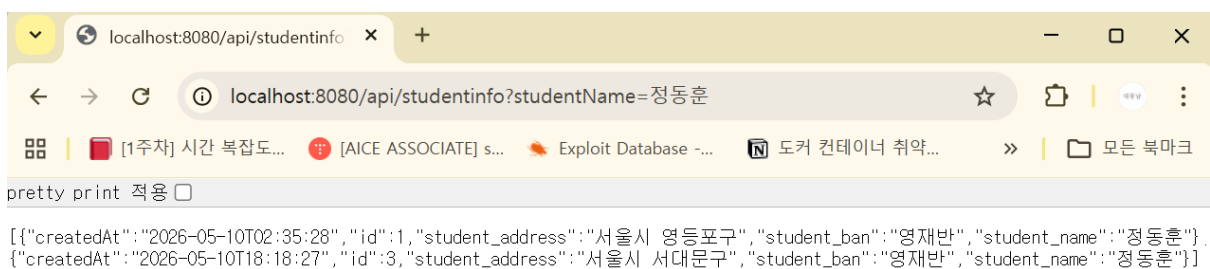
## 4-1 디지털대성 API 하나를 Spring Boot로 마이그레이션

학생 정보 조회를 선택해서 만들어 봤습니다.

기능은 등록, 조회, 삭제가 가능하게 하였고 이름을 검색하면 중복인 학생들도 있으니 중복되게 검색 가능하게 하였고, 없을 시 전체 학생이 출력되게 하였습니다. 삭제인 경우에는 id를 넣으면 삭제가 가능하게 하였습니다.



## 크롬 url에서 확인한 전체 조회(findall())



studentName 파라미터 적용

```

@GetMapping
public ResponseEntity<List<StudentinfoResponse>> get(@RequestParam(required = false) String studentName){
    if(studentName != null){
        return ResponseEntity.ok(studentinfoService.findByStudent(studentName));
    }
    return ResponseEntity.ok(studentinfoService.findAll());
}

```

## StudentinfoController Entity 구성 로직

```

<select id="findById" resultType="Studentinfo">
    SELECT id, student_name, student_address, student_ban, created_at
    FROM studentinfo
    WHERE id = #{id}
</select>

<select id="findByStudent" resultType="Studentinfo">
    SELECT id, student_name, student_address, student_ban, created_at
    FROM studentinfo
    WHERE student_name = #{studentName}
</select>

```

## StudentinfoMapper 쿼리 구성

```

<delete id="deleteById">
    DELETE FROM studentinfo WHERE id = # {id}
</delete>

```

## StudentinfoMapper 삭제 쿼리

```

@Transactional 1개 사용 위치
public void deleteById(Long id) {studentinfoMapper.deleteById(id);
}

```

StudentinfoService delete 적용 @Transactional 어노테이션 적용

## 자습과제

### 4-2 Spring XML과 Spring Boot 자동 설정 차이점 정리

정확하게 실무에서 했던 것들이 기억이 안나서 인터넷에서 보고 적어보았습니다!

#### 4-2-1 XML 설정 방식

모든 객체의 생성과 관계 설정을 외부 XML 파일에 기술하는 방식입니다.

단점으로는 프로젝트가 커지면 XML 관리 난이도가 급상승합니다.

특징으로는 컴파일 없이 설정만 변경하여 애플리케이션 동작을 바꿀 수 있습니다.

Ex)

```
<bean id="userService" class="com.example.UserService">
    <property name="userRepository" ref="userRepository" />
</bean>
```

어노테이션 기반 설정은 자바 코드 위에 @Component, @Service, @Autowired 등의 어노테이션을 붙여 프레임워크에 지시를 내리는 방식입니다.

환경설정 진화: Properties → YAML

Properties 방식은

키-값 형태로 중복된 키 이름을 반복해서 적어야 했습니다.

YAML 방식은

계층 구조를 사용하여 데이터 간의 관계를 알아보기 쉬워졌습니다.

#### 4-2-2 의존성 관리 (개별 의존성 → Starter)

개별 의존성 관리는 Spring-web, spring-webmvc, Jackson-databind, tomcat-embed-core 등 수십 개의 의존성을 일일이 버전까지 적어가며 추가해야 했습니다.

Starter 의존성 관리는 [springboot.io](https://springboot.io)에서 설정을 해주면 starter-boot-starter-spring-webmvc 같은 한 줄의 선언으로 관련된 모든 라이브러리를 가져옵니다.

버전 관리는 pom이 최적 버전을 관리한다고 합니다.

#### 4-2-3

과거에는 외부 WAS(Tomcat) + WAR

서버 컴퓨터에 별도로 Tomcat(WAS)를 설치하고 환경 설정을 합니다.

개발한 소스코드를 WAR 파일로 압축합니다.

현재에 springboot 에서는

애플리케이션 내부에 Tomcat 라이브러리가 포함 되어 있습니다.

전체 프로젝트를 하나의 JAR 파일로 빌드합니다.

도커나 컨테이너 환경이나 클라우드 서비스에 배포하기 매우 유리합니다.

#### 4-3 기존 Spring 프로젝트 구조 vs Spring Boot 프로젝트 구조 비교

기존 Spring 설정 파일이 Boot 구조에서는 보이지 않습니다.

web.xml (Deployment Descriptor):

웹 애플리케이션의 시작점입니다.

서블릿 등록, 필터 설정, 에러 페이지 등을 정의하던 파일입니다.

dispatcher-servlet.xml (Servlet Context):

웹 계층(Controller, ViewResolver 등)에 필요한 빈을 정의하던 곳입니다.

applicationContext.xml (Root Context):

서비스, 데이터베이스(DB), 보안 등 비즈니스 로직과 관련된 공통 빈을

설정하던 곳입니다.

**파일들은 사라진 것이 아니라 자바 코드 안으로 숨거나 자동화되었습니다.**

**설정의 자바화 (@Configuration):** XML에 적던 빈(Bean) 등록 정보는 이제 자바 클래스 위에 @Configuration과 @Bean 어노테이션을 붙여 처리합니다. 이제 IDE의 자동 완성 기능을 활용해 오타 없이 설정을 짤 수 있게 되었습니다.

**전략적 통합 (application.yml / application.properties):**

복잡한 XML 태그 대신,

직관적인 계층 구조를 가진 YAML 파일 하나에 DB 연결 정보나

서버 포트 설정을 모아놓습니다.

**(@SpringBootApplication):** 메인 클래스에 붙은 이 어노테이션 하나가 web.xml과 applicationContext.xml의 역할을 대신합니다.

이 어노테이션 내부의 @EnableAutoConfiguration이 프로젝트에 추가된 라이브러리(Starter)를 보고 필요한 설정을 알아서 다 해줍니다.

### 3. 왜 사라졌는가?

스스로 서버를 띄우니 외부 서버용 설정이 필요 없어졌고, 미리 웹과 톰캣을 미리 준비하여 애플리케이션 내에서 제공하며 복잡한 XML 태그 대신 (빈등록)직관적인 계층 구조를 가진 YAML 파일 하나에 DB연결 정보나 서버 포트 설정을 모아 넣었습니다.

## Spring Boot가 Spring과 뭐가 다른가?

### 1) 자동설정

클래스패스(=pom.xml에 추가된 의존성이 빌드 후 실제 실행 환경에 올라와 있는지)를 보고 필요한 설정을 알아서 등록해주는 기능.

Spring-boot-starter-webmvc 의존성 하나만 추가하면 → 톰캣, DispatcherServlet, Jackson, 기본 에러 페이지가 자동 등록

디지털대성에서 작성했던 web.xml, dispatcher-servlet.xml 작업이 0줄로 줄어들

### 2) 내장 톰캣

톰캣을 따로 설치할 필요 없이 애플리케이션 jar안에 함께 묶임.

Java -jar app.jar 한 줄로 서버 실행

클라우드(Docker,Kubernetes)에 적합

### 3) Starter 의존성

관련 라이브러리들을 묶어서 호환 버전 셋으로 한 번에 가져오는 메타 의존성.

버전 정합성은 Spring Boot의 BOM(Bill of Materials)이 관리

예: spring-boot-starter-webmvc 하나로 톰캣 + Spring MVC + Jackson + Logback이

검증된 호환 버전으로 한꺼번에 들어옴

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
```

Spring-boot-starter-webmvc 확인

패키지 구조 만들기

@SpringBootApplication이 붙은 메인 클래스의 패키지(com.dsa.boardapi) 하위만 컴포넌트 스캔됩니다. 메인 클래스를 최상위에 두고 그아래에 board, controller 같은 하위 패키지를 두는 게 정석, 빈을 못 찾는 예러가 나면 가장 먼저 패키지 위치부터 확인하세요.

궁금한점 정리:

자바에서 Optional<T>를 사용하는 가장 핵심적인 이유는 **'값이 없을 수도 있음'을 명시적으로 표현하여 NullPointerException(NPE)을 방지**하기 위함입니다.

단순히 null을 체크하는 것을 넘어, 코드의 의도와 안정성을 높여주는 구체적인 이유는 다음과 같습니다.